

Transformações Bidimensionais

Prof. Me João Paulo Biazotto

Universidade Estadual do Paraná – Campus Apucarana
Bacharelado em Ciência da Computação

Computação Gráfica
2026



Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação
- 4 Escala
- 5 Rotação
- 6 Matrizes e coordenadas homogêneas
- 7 Composição
- 8 Aplicação com Python e OpenGL
- 9 Atividade
- 10 Fechamento



Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação
- 4 Escala
- 5 Rotação
- 6 Matrizes e coordenadas homogêneas
- 7 Composição
- 8 Aplicação com Python e OpenGL
- 9 Atividade
- 10 Fechamento



Por que estudar transformações?

Em Computação Gráfica, uma figura é representada por pontos.

Ideia central

Transformar uma figura significa transformar os pontos que compõem essa figura.

- Mover uma figura: alterar suas coordenadas.
- Aumentar ou diminuir: multiplicar coordenadas.
- Girar: recalcular coordenadas usando um ângulo.

Uma figura como conjunto de pontos

Considere um triângulo:

$$A = (1, 1), \quad B = (3, 1), \quad C = (2, 3)$$

Para transformar o triângulo, transformamos:

$$A \rightarrow A', \quad B \rightarrow B', \quad C \rightarrow C'$$

Logo:

O OpenGL desenha vértices. Se os vértices mudam, a figura muda.



Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação
- 4 Escala
- 5 Rotação
- 6 Matrizes e coordenadas homogêneas
- 7 Composição
- 8 Aplicação com Python e OpenGL
- 9 Atividade
- 10 Fechamento



Ponto

$$P = (x, y)$$

Indica uma posição no plano.

Vetor

$$\vec{v} = (v_x, v_y)$$

Indica deslocamento, direção e sentido.

Interpretação

Um ponto pode ser visto como o final de um vetor que começa na origem.

Transformação geométrica

Uma transformação recebe um ponto e produz outro ponto:

$$T(P) = P'$$

Ou:

$$(x, y) \rightarrow (x', y')$$

- Translação
- Escala
- Rotação



Transformação afim

Uma transformação afim pode ser descrita como:

$$x' = Ax + b$$

- Ax : transformação linear.
- $+b$: deslocamento ou translação.

Assim:

Transformações afins preservam linhas retas.

Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação**
- 4 Escala
- 5 Rotação
- 6 Matrizes e coordenadas homogêneas
- 7 Composição
- 8 Aplicação com Python e OpenGL
- 9 Atividade
- 10 Fechamento



Translação

Transladar significa mover todos os pontos pela mesma quantidade.

$$x' = x + \Delta x$$

$$y' = y + \Delta y$$

Interpretação

Δx desloca horizontalmente; Δy desloca verticalmente.

Exemplo de translação

Considere:

$$P = (2, 3)$$

e:

$$\Delta x = 4, \quad \Delta y = -1$$

Então:

$$x' = 2 + 4 = 6$$

$$y' = 3 - 1 = 2$$

Logo:

$$P' = (6, 2)$$



O que a translação preserva?

A translação altera:

- posição.

Mas preserva:

- tamanho;
- distâncias;
- ângulos;
- orientação.

Conceito

Translação é uma operação isométrica.



Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação
- 4 Escala**
- 5 Rotação
- 6 Matrizes e coordenadas homogêneas
- 7 Composição
- 8 Aplicação com Python e OpenGL
- 9 Atividade
- 10 Fechamento



Escalar significa multiplicar as coordenadas por fatores.

$$x' = xE_x$$

$$y' = yE_y$$

- E_x : fator de escala no eixo x .
- E_y : fator de escala no eixo y .

Exemplo de escala

Considere:

$$P = (2, 3)$$

e:

$$E_x = 2, \quad E_y = 2$$

Então:

$$x' = 2 \cdot 2 = 4$$

$$y' = 3 \cdot 2 = 6$$

Logo:

$$P' = (4, 6)$$



Escala uniforme e não uniforme

Escala uniforme

$$E_x = E_y$$

Mantém proporções.

Escala não uniforme

$$E_x \neq E_y$$

Pode deformar a figura.

Escala em relação à origem

A escala básica ocorre em relação à origem:

$$(0,0)$$

Se o objeto não estiver centralizado na origem, a escala pode parecer também deslocar o objeto.

Ponto importante

O centro da transformação importa.

Escala em relação ao centro

Para escalar em relação ao centro C :

- 1 transladar o centro para a origem;
- 2 aplicar escala;
- 3 transladar de volta.

$$P_1 = P - C$$

$$P_2 = S(P_1)$$

$$P_3 = P_2 + C$$



Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação
- 4 Escala
- 5 Rotação**
- 6 Matrizes e coordenadas homogêneas
- 7 Composição
- 8 Aplicação com Python e OpenGL
- 9 Atividade
- 10 Fechamento



Rotacionar significa alterar a orientação de um ponto em torno de um centro.

Para rotação em torno da origem:

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

Exemplo: rotação de 90 graus

Considere:

$$P = (1, 0)$$

Para:

$$\theta = 90^\circ$$

temos:

$$\cos(90^\circ) = 0$$

$$\sin(90^\circ) = 1$$

Logo:

$$(1, 0) \rightarrow (0, 1)$$



Graus e radianos

Em sala, normalmente pensamos em graus.
Mas em Python:

$$\sin(\theta) \text{ e } \cos(\theta)$$

usam radianos.

$$\theta_{rad} = \theta_{graus} \frac{\pi}{180}$$

Python

```
angle = np.radians(90)
```



Rotação em torno do centro

A fórmula básica gira em torno da origem.

Para girar em torno do centro do objeto:

- 1 levar o centro até a origem;
- 2 rotacionar;
- 3 levar de volta.

$$P_1 = P - C$$

$$P_2 = R(P_1)$$

$$P_3 = P_2 + C$$



Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação
- 4 Escala
- 5 Rotação
- 6 Matrizes e coordenadas homogêneas**
- 7 Composição
- 8 Aplicação com Python e OpenGL
- 9 Atividade
- 10 Fechamento



Por que usar matrizes?

Matrizes permitem:

- representar transformações de forma padronizada;
- combinar várias transformações;
- aplicar a mesma transformação a muitos pontos;
- enviar transformações para shaders.



Matriz de escala

$$S = \begin{bmatrix} E_x & 0 \\ 0 & E_y \end{bmatrix}$$

Aplicando a um ponto:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} E_x & 0 \\ 0 & E_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Resultado:

$$x' = E_x x, \quad y' = E_y y$$

Matriz de rotação

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Resultado:

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$



Coordenadas homogêneas

Usamos:

$$(x, y, 1)$$

em vez de apenas:

$$(x, y)$$

Por quê?

Para representar translação, escala e rotação como matrizes 3×3 .



Matriz de translação

$$T = \begin{bmatrix} 1 & 0 & \Delta x \\ 0 & 1 & \Delta y \\ 0 & 0 & 1 \end{bmatrix}$$

Aplicando:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = T \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Resultado:

$$x' = x + \Delta x, \quad y' = y + \Delta y$$

Translação

$$T = \begin{bmatrix} 1 & 0 & \Delta x \\ 0 & 1 & \Delta y \\ 0 & 0 & 1 \end{bmatrix}$$

Escala

$$S = \begin{bmatrix} E_x & 0 & 0 \\ 0 & E_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Matriz de rotação 3×3

$$R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Ideia

A terceira coordenada permite usar a mesma estrutura para todas as transformações.

Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação
- 4 Escala
- 5 Rotação
- 6 Matrizes e coordenadas homogêneas
- 7 Composição**
- 8 Aplicação com Python e OpenGL
- 9 Atividade
- 10 Fechamento



Composição de transformações

Composição significa aplicar transformações em sequência.

$$P' = T_1(T_2(P))$$

Ou, com matrizes:

$$P' = M_1 M_2 P$$

Ideia prática

Uma única matriz pode representar várias transformações combinadas.



A ordem importa

Em geral:

$$AB \neq BA$$

Portanto:

$$TR \neq RT$$

Importante

Transladar e depois rotacionar pode produzir resultado diferente de rotacionar e depois transladar.



Exemplo: ordem das transformações

Ponto:

$$P = (1, 0)$$

Caso A

Transladar por $(1, 0)$ e depois rotacionar 90° :

$$(1, 0) \rightarrow (2, 0) \rightarrow (0, 2)$$

Caso B

Rotacionar 90° e depois transladar por $(1, 0)$:

$$(1, 0) \rightarrow (0, 1) \rightarrow (1, 1)$$

$$(0, 2) \neq (1, 1)$$



Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação
- 4 Escala
- 5 Rotação
- 6 Matrizes e coordenadas homogêneas
- 7 Composição
- 8 Aplicação com Python e OpenGL**
- 9 Atividade
- 10 Fechamento



Matriz de translação em Python

```
def translation_matrix(dx, dy):  
    return np.array([  
        [1.0, 0.0, dx],  
        [0.0, 1.0, dy],  
        [0.0, 0.0, 1.0]  
    ], dtype=np.float32)
```

Matriz de escala em Python

```
def scale_matrix(sx, sy):  
    return np.array([  
        [sx, 0.0, 0.0],  
        [0.0, sy, 0.0],  
        [0.0, 0.0, 1.0]  
    ], dtype=np.float32)
```

Matriz de rotação em Python

```
def rotation_matrix(angle_degrees):  
    angle = np.radians(angle_degrees)  
    c = np.cos(angle)  
    s = np.sin(angle)  
  
    return np.array([  
        [c, -s, 0.0],  
        [s,  c, 0.0],  
        [0.0, 0.0, 1.0]  
    ], dtype=np.float32)
```



Vertex Shader com matriz

```
#version 330 core

layout(location = 0) in vec2 aPos;

uniform mat3 uTransform;

void main()
{
    vec3 p = vec3(aPos, 1.0);
    vec3 transformed = uTransform * p;
    gl_Position = vec4(transformed.xy, 0.0, 1.0);
}
```



Fragment Shader com cor

```
#version 330 core

out vec4 FragColor;

uniform vec3 uColor;

void main()
{
    FragColor = vec4(uColor, 1.0);
}
```



Enviando a matriz ao shader

```
transform_location = glGetUniformLocation(  
    shader_program,  
    "uTransform"  
)  
  
glUseProgram(shader_program)  
  
glUniformMatrix3fv(  
    transform_location,  
    1,  
    GL_TRUE,  
    transform_matrix  
)
```

Compondo a matriz final

```
T = translation_matrix(tx, ty)
R = rotation_matrix(angle)
S = scale_matrix(scale, scale)

M = T @ R @ S
```

$$MP = T(R(S(P)))$$

Leitura

A transformação mais à direita é aplicada primeiro.



Comandos de teclado

Tecla	Ação
Setas	Transladar
r/R	Rotacionar horário/anti-horário
e/E	Aumentar/diminuir escala
Espaço	Resetar transformação
Esc	Sair

Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação
- 4 Escala
- 5 Rotação
- 6 Matrizes e coordenadas homogêneas
- 7 Composição
- 8 Aplicação com Python e OpenGL
- 9 Atividade
- 10 Fechamento



Desenvolver uma aplicação OpenGL que desenhe uma figura 2D e permita transformá-la pelo teclado.

- Setas: translação.
- r/R: rotação.
- e/E: escala.
- Espaço: resetar.

Restrição

Não utilizar funções prontas como `glTranslate` ou `glRotate`.

Requisitos da atividade

- 1 Usar VAO e VBO.
- 2 Usar Vertex Shader e Fragment Shader.
- 3 Criar matrizes 3×3 manualmente.
- 4 Enviar transformação por `uniform mat3`.
- 5 Implementar comandos de teclado.
- 6 Explicar a ordem das transformações usada.

Agenda

- 1 Motivação
- 2 Vetores e transformações afins
- 3 Translação
- 4 Escala
- 5 Rotação
- 6 Matrizes e coordenadas homogêneas
- 7 Composição
- 8 Aplicação com Python e OpenGL
- 9 Atividade
- 10 Fechamento



Transformações bidimensionais permitem alterar pontos de uma figura.

- Translação: soma deslocamentos.
- Escala: multiplica coordenadas.
- Rotação: usa seno e cosseno.
- Matrizes: padronizam as transformações.
- Coordenadas homogêneas: permitem representar translação como matriz.
- Composição: combina várias transformações.

Mensagem final

Ideia central

Transformar uma figura é aplicar uma regra matemática aos seus pontos.

Computação Gráfica

O OpenGL desenha os vértices transformados.

